

小程序 SDK 接口说明文档 V3.0.4

文档修改记录

序号	版本号	修改内容	修改者	修改日期
1	v1.0.0	文档建立	虞明义	2022/3/12
2	V1.1.0	功能新增：1. 新机型支持 B1、B2032。条形码支持由原来的 code128C 变为为 CODE128 全量支持（CODE128A、CODE128B、CODE128C、CODE128AUTO） 3. 二维码中文支持 BUG 修复：1. 部分场景极低概率连接异常	虞明义	2022/9/8
3	v1.1.3	新增适配 A8/A20/A63 等机型	虞明义	2022/12/27
4	v1.1.4	功能新增：1. 文本绘制接口更新，支持文本加粗 2. 一维码绘制接口更新，支持一维码可识别文字展示	虞明义	2022/12/27
5	v1.1.5	功能新增：绘制文字增加字体，斜体控制，字体仅支持微信内置字体，iOS 不支持中文斜体、IOS 字体不支持中文 Bug 修复： 【B32-V7.10】【必现】使用	虞明义	2023/2/17

		iPhone7，无法打印高度超过110mm 的模板】		
6	v1.1.6	功能新增：1. 企业微信小程序兼容 2.uni-app js 调用支持	虞明义	2023/3/3
7	V1.1.7	1.导出模式统一兼容 Uni-app Vue2 和 Vue3 调用	虞明义	2023/6/25
8	V1.1.8	功能新增：1. 支持 K3/K3W 蓝牙打印 Bug 修复：1. 优化手机蓝牙兼容性 2. 修复 BUG 若干	虞明义	2023/8/11
9	V1.1.8	功能新增：1. 支持 M2 机型蓝牙打印	虞明义	2023/11/17
10	V1.1.9	功能新增：1. 支持第三方蓝牙设备与打印机同时连接 2. 支持 B3S_P 以及 B21_H 机型	虞明义	2024/3/14
11	V3.0.0	功能新增：支持设置 API 支持平台，包含微信、钉钉、飞书（飞书平台仅支持 B31）支持获取链接状态正式支持获取已连接设备 SN 号、已连接设备	虞明义	2024/9/21
12	V3.0.1	修复 BUG：1. 钉钉升级到最新版本后，连接成功后切换页面后会自动断开连接 且无法再次连接	虞明义	2025-02.11
13	V3.0.2	修复 BUG：1. 微信小程序历史通过 VUE2 接入完成客户	虞明义	2025.03.08

		反馈安卓手机打印时绘制超时异常无法正常打印调整接口：1. 新增设置编译平台接口 2.startDrawLabel 新增 canvas 对象参数		
14	V3.0.3	修复 BUG：1.uni 编译环境下，部分手机微信小程序打印黑块问题 2。微信小程序 iOS 蓝牙发送数据模式修改，提升发送速率（微信小程序基础库蓝牙功能更新兼容）3。企业微信连接优化 新功能：1。支持绘制文字自动换行	虞明义	2025.03.24
15	V3.0.4	优化：1。微信及企业微信连接优化 2。绘制优化 3.UNI-DEMO 支持更多打印示例新功能：1。支持图片打印 2.M2 机型支持获取纸张信息	虞明义	2025.06.18
16	V3.0.4	BUG 修复：修复打印网络图片、Base64 有黑块	虞明义	2025.06.30

产品目的

JCAPI 接口方法说明文档，是针对标签绘图打印提出的接口方法说明，方便用户在二次开发中调用接口，缩短开发周期，加快开发进度

打印机支持

--	--	--	--

打印机型号	打印机硬件版本	打印机固件版	支持本小程序类型
B3S/A8	V35.0	V35.18 及以上版本	微信、钉钉
B3S_P	全版本	全版本	微信、钉钉
B1	全版本	全版本（部分版本可能需 要进行固件升级）	微信、钉钉
B203/ A20	全版本	全版本（部分版本可能需 要进行固件升级）	微信、钉钉
B21	V40	V40 及以上版本	微信、钉钉、飞书
B21_H	全版本	全版本	微信、钉钉、飞书
B21_Pro	全版本	全版本	微信、钉钉、飞书
B31	全版本	全版本	微信、钉钉、飞书
B4	全版本	全版本	微信、钉钉、飞书
K2	全版本	全版本	微信、钉钉、飞书
K3/K3W	V2	V2.12 及以上版本	微信、钉钉
Z401	V6.0	V6.05 及以上版本	微信、钉钉
B32/A63/B32R	V7.0	V7.05 及以上版本	微信、钉钉
M2_H	全版本	全版本	微信、钉钉
M3_H	全版本	全版本	微信、钉钉、飞书

一、使用说明

1.1 工程配置说明

1. 本打印接口需运行在小程序上（微信、飞书、钉钉）。
2. 将“JCAPI”文件夹，放到工程的“utils”目录下。（..\Demo\utils\JCAPI）
3. 将“app.js”文件中的“globalData”对象复制到工程的“app.js”中。
4. 必须在布局文件中使用 **canvas 控件**，才能实现绘图并读取图片数据，如下：

代码块

```
1 <!--不可隐藏，如不展示canvas，可以把坐标改为9999-->
```

```

2    <view class="show canvas">
3        <!--宽高建议用变量-->
4        <canvas canvas-id="test" style="width:{{canvasWidth}}px; height:
        {{canvasHeight}}px;" />
5    </view>

```

5. canvas 画布的宽高需要与 startDrawLabel 中的宽高按比例对应，画布的宽高单位为 px，startJob 中的宽高单位为 mm（注：当 canvas 的宽高小于 startDrawLabel 预设的宽高时，打印会产生黑块），且 canvas id 需要与 startDrawLabel 对应（推荐使用方式 2）

a. 方式 1：基于打印机的分辨率将尺寸固定写死

- i. 200dpi 机型 1mm=8px，300dpi 机型 1mm=11.81px。（B3S/B1//B203/K3/K3W 为 200dpi，Z401/B32/B32R 为 300dpi）
- ii. 例如打印机为 B3S，标签尺寸为宽 50mm、高 40mm，示例如下：

代码块

```

1    <!--不可隐藏，如不展示canvas，可以把坐标改为9999-->
2    <view class="show canvas">
3        <!--宽高建议用变量-->
4        <canvas canvas-id="test" style="width:400px; height:320px;" />
5    </view>

```

代码块

```

1    //绑定canvas，id传入数据与canvas id一致，标签设置物理尺寸
2    JCAPI.startDrawLabel('test', this, 50, 40, 0);

```

2. 方式 2：宽高使用变量让 SDK 自动调节示例调用在"..\\Demo\\pages\\index"下的"index.js"、"index.wxml"文件（以微信小程序为例）

代码块

```

1    <!--不可隐藏，如不展示canvas，可以把坐标改为9999-->
2    <view class="show canvas">
3        <!--宽高建议用变量-->
4        style="width:{{canvasWidth}}px; height:{{canvasHeight}}px;"
5    </view>

```

代码块

```

1    Page({
2        //设定canvas初始值

```

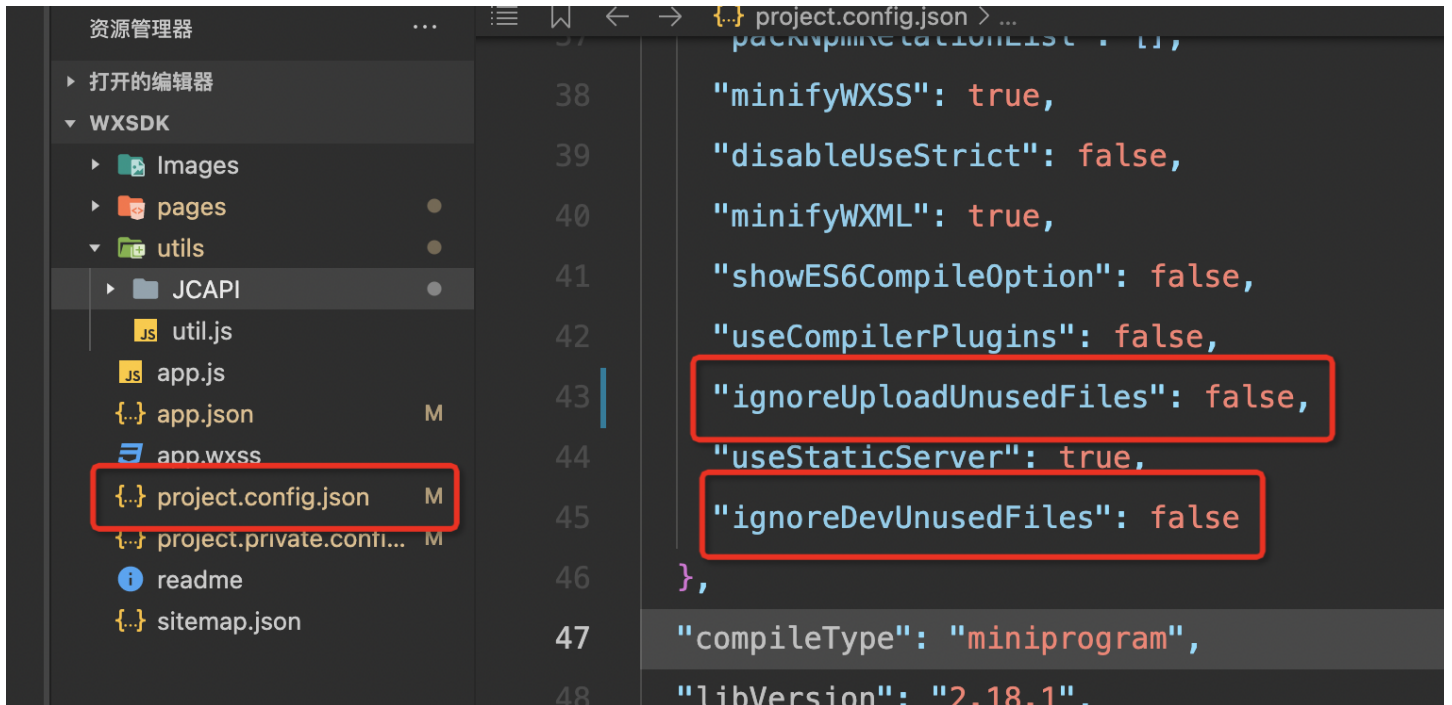
```

3     data: {
4         canvasWidth: 320,
5         canvasHeight: 200,
6     }
7 })
8
9 //绑定canvas, id传入数据与canvas id一致, 标签设置物理尺寸
10 JCAPI.startDrawLabel('test', this, 50, 40, 0);

```

6. 示例调用在"..\\Demo\\pages\\index"下的"index.js"、"index.wxml"文件（以微信小程序为例）

7. 工程配置，需进行如下配置，否则编译时会找不到被混淆的文件（以微信小程序为例）



1.2 微信权限配置及手机设置说明

Android 端配置：需要开启手机蓝牙、GPS 定位、授予微信位置权限（以微信小程序为例）

IOS 端配置：需要开启手机蓝牙、授予微信位置权限、蓝牙权限（以微信小程序为例）



1.3 不兼容场景说明

1.3.1 部分手机不支持后台打印

1.部分安卓手机系统和 IOS 系统限制，应用后台使用蓝牙发送数据存在限制，会造成数据丢失的情况。故小程序仅**支持处于前台时进行打印**

1.3.2 部分品牌手机不能配对

1.小米、红米、一加等品牌的部分系统版本存在系统设置里面进行蓝牙配对后，无法在微信小程序中进行连接的问题（小程序使用的是 **BLE 低功耗蓝牙，无需进行蓝牙配对**）

1.3.3 项目配置不能勾选代码保护

1.SDK 的部分代码已经经过混淆，如果勾选代码保护将会编译出错

1.4 打印相关功能调试建议

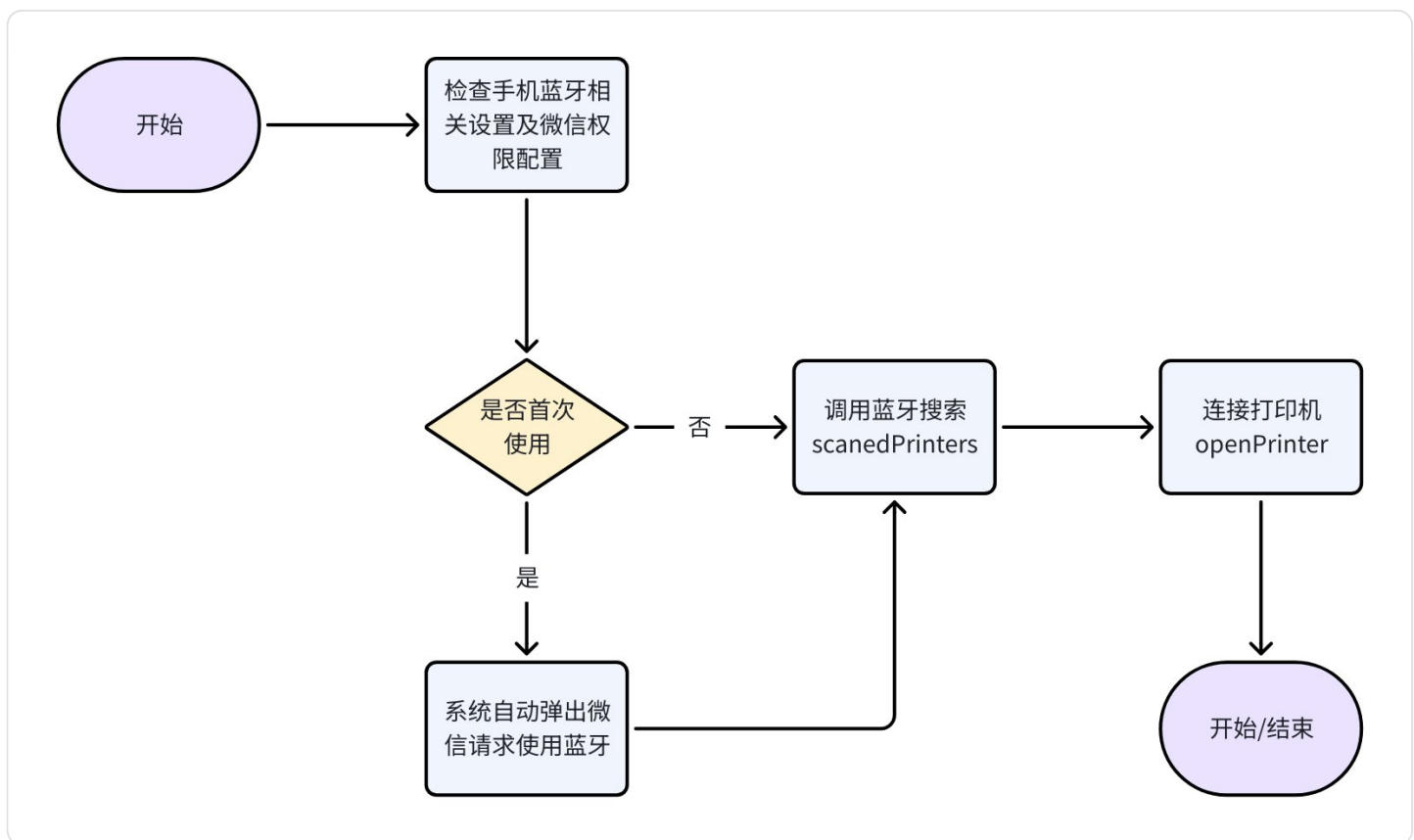
- 建议在调试打印相关功能时，不要运行真机调试。因为真机调试会影响蓝牙传输速度，导致打印异常、超时等现象。（以微信小程序为例）



- 建议在调试打印功能时，通过预览运行到真机，并配合日志进行功能调试。

二、样例演示

2.1 连接打印机流程（以微信小程序为例）



2.1.1 搜索打印机

代码块

```
1 // 搜索打印机
2 scanPrinter(){
```



```

3    let self = this;
4    // 显示加载提示
5    wx.showLoading({
6      title: '搜索中',
7      mask: true
8    })
9    // 使用 JCAPI.scannedPrinters 函数搜索附近的蓝牙设备，传入一个回调函数作为参数
10   JCAPI.scannedPrinters((didGetScannedPrinters) => {
11     // 隐藏加载提示
12     wx.hideLoading()
13     // 创建一个新的数组 arr 并将 didGetScannedPrinters 中的内容复制给它
14     let arr = didGetScannedPrinters
15     // 过滤打印机列表，只保留名称中不包含 "未知" 的打印机
16     arr = didGetScannedPrinters.filter(items => {
17       return items.name.indexOf('未知')<0;
18     })
19     // 检查是否存在当前选定的打印机 (self.data.nowPrint 是当前选定的打印机)
20     if(self.data.nowPrint&&self.data.nowPrint.name){
21       // 如果存在当前选定的打印机，再次过滤打印机列表，将与当前选定打印机的名称不匹配的打
打印机保留下来
22       arr = arr.filter(items => {
23         return items.name !== self.data.nowPrint.name
24       });
25     }
26     // 使用 this.setData() 更新小程序页面的数据：
27     // - 设置 showPrinters 为 true，以显示打印机列表
28     // - 设置 printerLists 为经过过滤后的打印机列表 arr，用于在页面上显示这些打印机
29     this.setData({
30       showPrinters:true,
31       printerLists: arr
32     })
33   })
34 },

```

2.1.2 连接打印机

代码块

```

1  // 连接打印机
2  openPrinter(e) {
3    let item = {};
4    // 如果本地存储中存在打印机信息，则将其存储在变量 item 中
5    if (wx.getStorageSync('print')) {
6      item = wx.getStorageSync('print');
7    }

```

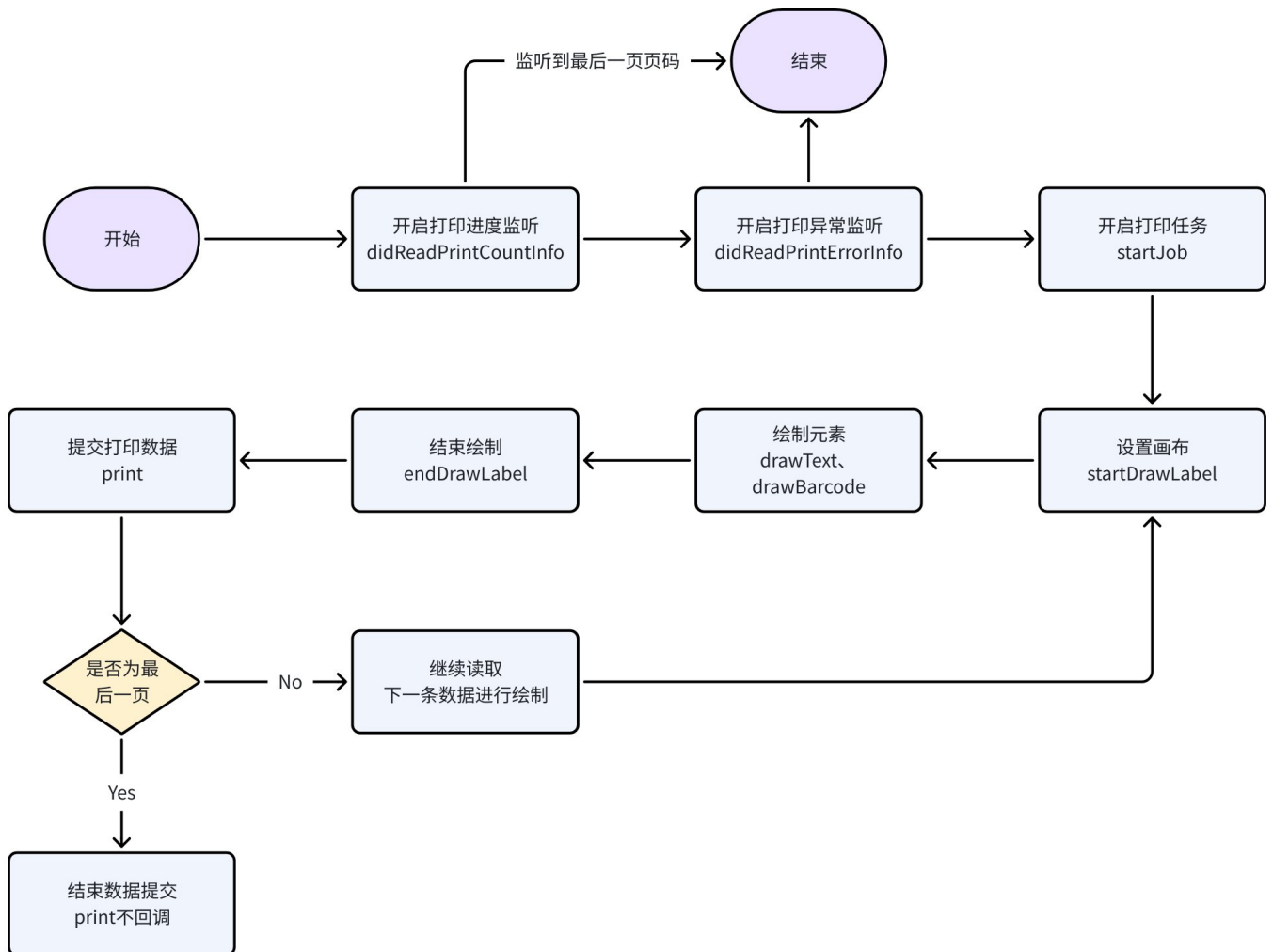
```
8    // 如果事件对象 e 存在，并且包含目标数据项（通常是用户点击了某个打印机项），则将该数据
    项存储在变量 item 中
9    if (e && e.currentTarget.dataset.item) {
10        item = e.currentTarget.dataset.item;
11    }
12
13    let self = this;
14    // 显示加载提示，标题为 "连接中"
15    wx.showLoading({
16        title: '连接中',
17        mask: true
18    });
19
20    // 调用 JCAPI.openPrinter 函数来尝试连接打印机
21    JCAPI.openPrinter(item.name, function () {
22        // 连接成功后的回调函数
23        wx.showToast({
24            title: '连接打印机成功',
25            icon: '',
26            image: '',
27            duration: 2000,
28            mask: true,
29            success: function (res) {
30                // 隐藏加载提示
31                wx.hideLoading();
32
33                // 从当前的打印机列表中移除已连接的打印机
34                let Printer = self.data.printerLists;
35                var arr = Printer.filter(function (items) {
36                    return items.name !== item.name;
37                });
38
39                // 将连接成功的打印机信息存储在本地存储中
40                wx.setStorageSync('print', item);
41
42                // 更新小程序页面的数据，设置当前打印机为连接成功的打印机，并更新打印机列表
43                self.setData({
44                    nowPrint: item,
45                    printerLists: arr
46                });
47            },
48        });
49    }, function () {
50        // 连接失败后的回调函数
51        // 清除当前的打印机信息
52        self.setData({
53            nowPrint: ""
```

```

54     });
55     wx.removeStorageSync("print");
56
57     // 显示连接失败的提示
58     wx.showToast({
59       title: '连接失败',
60       icon: 'none',
61       duration: 2000
62     });
63   });
64 },

```

2.2 打印流程



2.2.1 打印示例

代码块

```
1 //打印方法示例
```

```
2  print(e) {
3      // 检查是否有要打印的内容
4      if (this.data.pageModels.length == 0) {
5          wx.showToast({
6              title: '请添加打印内容',
7              icon: 'none',
8              duration: 2000
9          });
10         return;
11     }
12
13     // 检查是否已选择打印机, 如果没有则触发扫描打印机操作
14     if (!this.data.nowPrint || !this.data.nowPrint.name) {
15         this.scanPrinter();
16         return;
17     }
18
19     // 将要打印的数据存储在本地存储中
20     wx.setStorageSync('lastPrintData', this.data.pageModels);
21
22     let count = 0;
23     let gapType = 0;
24     let darkness = 0;
25     let self = this;
26
27     // 获取要打印的总页数、纸张类型、浓度
28     for (let index = 0; index < this.data.pageModels.length; index++) {
29         let item = this.data.pageModels[index];
30         count += item.count;
31         if (index == 0) {
32             gapType = item.gapType;
33             darkness = item.darkness;
34         }
35     }
36
37     // 创建 Canvas 绘图上下文
38     let ctx = wx.createCanvasContext('test1', this);
39     ctx.setFillStyle('white');
40     ctx.fillRect(0, 0, 200, 80);
41     ctx.draw();
42     ctx.setFillStyle('black');
43     ctx.setFontSize(10);
44     ctx.fillText('准备打印', 3, 30);
45     ctx.draw();
46
47     // 步骤1: 监听打印进度
48     JCAPI.didReadPrintCountInfo(function (res) {
```

```
49     console.log("-----收到页面变化数据:-----");
50     console.log(res);
51     if (count == res.count) {
52         ctx.setFillStyle('black');
53         ctx.setFontSize(10);
54         ctx.fillText('打印完:' + res.count + '页', 3, 30);
55         if (res.tid) {
56             ctx.fillText('tid=' + res.tid, 50, 30);
57         }
58         ctx.draw();
59     } else {
60         ctx.setFillStyle('black');
61         ctx.setFontSize(10);
62         ctx.fillText('打印了:' + res.count + '页', 3, 30);
63         if (res.tid) {
64             ctx.fillText('tid=' + res.tid, 50, 30);
65         }
66         ctx.draw();
67     }
68 });
69
70 // 步骤2: 监听打印错误信息
71 JCAPI.didReadPrintErrorInfo(function (res) {
72     console.log("-----收到出错信息 :-----");
73     console.log(res);
74     ctx.setFillStyle('black');
75     ctx.setFontSize(10);
76     ctx.fillText('错误:' + res.errCode + ' ' + res.msg, 3, 30);
77     ctx.draw();
78 });
79
80 // 初始化计数器
81 let jishu = 0;
82
83 //步骤3: 开启打印任务
84 JCAPI.startJob(gapType, darkness, count, function () {
85     self.startPrinting(jishu);
86 });
87 },
88
89 // 打印内容
90 startPrinting(jishu) {
91     if (this.data.pageModels.length <= jishu) return;
92     let self = this;
93     let item = this.data.pageModels[jishu];
94     let pageCount = item.count;
95 }
```

```

96 //步骤4: 开始绘制标签
97 JCAPI.startDrawLabel('test', this, item.w, item.h, item.rotation);
98 let elements = item.elements;
99
100 // 步骤5: 逐个绘制页面元素
101 for (let i = 0; i < elements.length; i++) {
102     let element = elements[i];
103     if (element.id == 1) {
104         JCAPI.drawText(element.value, element.x, element.y, element.fontSize,
105             element.rotation, element.options);
106     } else if (element.id == 2) {
107         JCAPI.drawLine(element.x, element.y, element.w, element.h,
108             element.rotation);
109     } else if (element.id == 3) {
110         JCAPI.drawBarcode(element.value, element.x, element.y, element.w,
111             element.h, element.rotation, element.fontSize, element.fontHeight,
112             element.fontPostion);
113     } else if (element.id == 4) {
114         JCAPI.drawQRCode(element.value, element.x, element.y, element.w,
115             element.h, element.rotation);
116     } else if (element.id == 5) {
117         JCAPI.drawRectangle(element.x, element.y, element.w, element.h,
118             element.linewidth, element.filled, element.rotation);
119     } else if (element.id == 6) {
120         JCAPI.drawImage(element.value, element.x, element.y, element.w,
121             element.h, element.rotation);
122     }
123 }
124
125 //仅B32R支持RFID打印,会使用到EPC功能, 其他打印机请忽略判断, 调用false部分
126 if (item.epc) {
127     // 步骤6: 结束绘制
128     JCAPI.endDrawLabel(function () {
129         // 步骤7: 提交打印数据,如果有 EPC 信息, 则使用 EPC 打印
130         JCAPI.print(pageCount, function () {
131             //最后一页数据提交, 不返回回调
132             jishu++;
133             self.startPrinting(jishu);
134         }, { 'epc': item.epc });
135     });
136 } else {
137     // 步骤6: 结束绘制, 正常打印
138     JCAPI.endDrawLabel(function () {
139         // 步骤7: 提交打印数据
140         JCAPI.print(pageCount, function () {
141             //最后一页数据提交, 不返回回调
142             jishu++;
143             self.startPrinting(jishu);
144         });
145     });
146 }

```

```
136     });  
137     });  
138   }  
139 },
```

三、接口列表

3.1 编译相关

3.1.1 设置小程序运行平台

代码块

```
1  /**  
2   * 设置小程序运行平台  
3   * @param {JCAPISDKPlatform} platform 平台:缺省微信  
4   */  
5  JCAPI.setPlatform=function(platform=JCAPISDKPlatform.wx)
```

3.1.2 设置代码编译平台

代码块

```
1  /**  
2   * 设置编译平台类型  
3   * @param {string} platform 支持: 原生环境编译、Uni环境编译, 缺省:原生环境编译  
4   */  
5  
JCAPI.setBuildPlatform(platform=JCAPISDKBuildPlatform.JCSDK_PLATFORM_Build_Origin)
```

3.2 第三方设备同时使用相关

3.2.1 设置是否使用第三方 BLE 监听

代码块

```
1  /**  
2   * 设置是否使用第三方BLE监听。需要在连接（openPrinter）之前使用  
3   * @param {boolean} use - 指定是否使用第三方BLE监听。默认为true。当程序多个使用蓝牙监听时，调用此方法，将监听外置出来供第三方调用  
4   */  
5  JCAPI.setUseThirdBleListen=function(use=true)
```

3.2.2 用于处理蓝牙特征值变化的事件（第三方设备同时使用时）

代码块

```
1  /**
2   * 当 setUseThirdBleListen 设置为 true 时有效。使用示例如下：
3   * ``javascript
4   * JCAPI.setUseThirdBleListen(true);
5   * wx.onBLECharacteristicValueChange((res) => {
6   *   JCAPI.bleValueChanged(res);
7   * });
8   * ``
9   * @param {*} res 蓝牙监听数据
10  */
11 JCAPI.bleValueChanged=function(res)
```

3.3 连接相关

3.3.1 搜索打印机

代码块

```
1  /**
2   * 搜索打印机
3   *
4   * @param {*} didGetScanedPrinters 搜索后的回调 function(list){}
5   */
6  JCAPI.scanedPrinters=function(didGetScanedPrinters)
```

3.3.2 连接打印机

代码块

```
1  /**
2   * 连接打印机
3   * @param {*} printerName 打印名称，如果为空字符串，则打开当前客户端系统上
4   * 的第一个支持的打印机。支持直接输入打印机型号。
5   * @param {*} didConnectedHandler 连接打印机的回调。
6   * @param {*} didDisconnectedHandler 断开打印机的回调。
7   */
8  JCAPI.openPrinter=function(printerName,didConnectedHandler,
9   didDisconnectedHandler)
```

3.3.3 关闭打印机连接

代码块**

```
2    * 关闭打印机连接
3    */
4    JCAPI.closePrinter=function()
```

3.3.4 获取已连接打印机 SN

代码块

```
1    /**
2    * 获取已连接打印机SN
3    * @param {JCSDKPrinterGetParas} callback 可选参数
4    */
5    JCAPI.getSN = function(callback)
```

3.3.5 获取已连接设备

代码块

```
1    /**
2    * 获取已连接设备
3    */
4    JCAPI.getConnName = function ()
```

3.4 信息获取相关

3.4.1 获取标签信息

代码块

```
1    /**
2    * 获取标签信息，目前仅支持M2
3    * @param {function} callback - 回调函数，用于返回标签信息。标签信息包含以下内容：
4    *     - 索引 0：间隙高度，单位像素
5    *     - 索引 1：标签高度（含间隙高度），单位像素
6    *     - 索引 2：纸张类型
7    *         - 1：间隙纸
8    *         - 2：黑标纸
9    *         - 3：连续纸
10    *         - 4：定孔纸
11    *         - 5：透明纸
12    *         - 6：标牌
13    *         - 10：黑标间隙纸
```

```

14  *   - 索引 3: 间隙高度, 单位毫米
15  *   - 索引 4: 标签高度 (含间隙高度), 单位毫米
16  *   - 索引 5: 标签宽度, 单位像素
17  *   - 索引 6: 标签宽度, 单位毫米
18  *   - 索引 7: 尾巴方向 (线缆标签、珠宝标签等)
19  *       - 1: 上
20  *       - 2: 下
21  *       - 3: 左
22  *       - 4: 右
23  *   - 索引 8: 尾巴长度, 单位像素
24  *   - 索引 9: 尾巴长度, 单位毫米
25  */
26  JCAPI.getPageInfos = function (callback)

```

3.5 绘制打印相关

3.5.1 开始绘制

代码块

```

1  /**
2   * 打印前开始绘制
3   * @param {string} canvasId - 画布ID
4   * @param {object} component - 画布所在js对象
5   * @param {number} labelWidth - 标签画布宽 (标签宽度, 单位mm)
6   * @param {number} labelHeight - 标签画布高 (标签高度, 单位mm)
7   * @param {number} rotation - 旋转角度, 默认为0, 支持0/90/180/270
8   * @param {Canvas} canvas - 画布对象。当使用原生小程序IDE时, `canvas` 可为 `null`;
   若通过调用 {@link setBuildPlatform} 方法将编译平台设置为 uni 编译时, 则需要传入
   `canvas` 对象, 否则打印会失败。
9   * @description
10  * 对应的 canvas 宽高建议使用变量
11  */
12  JCAPI.startDrawLabel = function (canvasId, compent, labelWidth, labelHeight,
   rotation = 0, canvas = null)

```

3.5.2 结束绘制

代码块

```

1  /**
2   * 结束绘制, 并在 canvas 生成标签图片。

```

```
3      *
4      * @param callback 绘制完之后的回调
5      *
6      */
7      JCAPI.endDrawLabel=function(callback)
```

3.5.3 绘制文本

3.5.3.1 绘制单行文本

代码块

```
1  /**
2   * 绘制单行文本
3   * @param {string} content 文本内容
4   * @param {number} x 起点x,单位mm
5   * @param {number} y 起点y,单位mm (左下角坐标)
6   * @param {number} fontHeight 字体大小,单位mm
7   * @param {number} rotation 旋转角度,默认为0,支持0/90/180/270,旋转中心点为左下角
8   * italic: ios不支持中文斜体
9   * @options {object} {bold:false,
10  *                  italic:false,
11  *                  family:'SimHei',
12  *                  align:'left'/'right'/'center'}
13  * 注意事项:
14  * 1. ios 斜体不支持中文,字体自定义也不支持中文,仅支持小程序内置字体。
15  * 2. 0° 对齐方式中心点为 X 坐标,即左对齐时文本左下角坐标为 x,右对齐时文本右下角坐标
16  *    为 x,居中对齐时文本底部中心点坐标为 x。
17  * 3. 90° 对齐方式中心点为旋转后的 Y 坐标,即左对齐时文本左下角坐标为旋转后的 Y 坐标,右
18  *    对齐时文本右下角坐标为旋转后的 Y 坐标,居中对齐时文本底部中心点坐标为旋转后的 Y 坐标。
19  * 4. 180° 和 270° 类似,可自行尝试。
20  */
21  JCAPI.drawText=function(content,x,y,fontHeight,rotation=0,options=null)
```

3.5.3.2 绘制多行文本

代码块

```
1  /**
2   * 绘制多行文本
3   * @param {string} content 文本内容
4   * @param {number} x 起点x,单位mm
5   * @param {number} y 起点y,单位mm (左下角坐标)
6   * @param {number} width 预设文本框宽度,单位mm
```

```

7    * @param {number} height 预设文本框高度, 单位mm
8    * @param {number} fontHeight 字体大小, 单位mm
9    * @param {number} rotation 旋转角度, 默认为0, 支持0/90/180/270, 旋转中心点为左下角
10   * @param {object} options 选项, 具体如下:
11   *   - bold: 是否加粗, 默认值为 false
12   *   - italic: 是否倾斜, 默认值为 false
13   *   - family: 字体设置, 默认值为 'SimHei'
14   *   - align: 对齐方式, 可选值为 'left'、'right'、'center'
15   *   - lineModel: 多行文本处理模式, 若 options 为空、不传 lineModel 或值不为 3, 则超出预设宽高的内容将被删除; 若值为 3, 则超出预设宽高的内容尾部将用省略号处理。使用多行文本时, options 参数不可为 null, 否则内容超出预设宽高时会异常 (下一版本修复)
16   *
17   * 注意事项:
18   * 1. iOS 斜体不支持中文, 字体自定义也不支持中文, 仅支持小程序内置字体。
19   */
20   JCAPI.drawTextInRect=function(content,x,y,width,height,fontHeight,rotation=0,options=null) {
21     manager.drawTextInRect(content,x,y,width,height,fontHeight,rotation,options);
22   }

```

3.5.4 绘制一维码

代码块

```

1    /**
2     * 绘制一维码
3     *
4     * @param {*} content 一维码内容
5     * @param {*} x 起点x, 单位:毫米。
6     * @param {*} y 起点y, 单位:毫米。
7     * @param {*} width 打印对象宽, 单位:毫米。
8     * @param {*} height 打印对象高, 单位:毫米。
9     * @param {*} rotation 旋转角度, 缺省值为0, 支持0/90/180/270, 旋转中心点左上角。
10    * @param {*} fontSize 文本的字体大小, 单位:毫米。
11    * @param {*} fontHeight 文本的高度, 单位:毫米。
12    * @param {*} postion 文本位置, 0-条码下方 1-条码上方
13    * 2-不显示文本, 此时fontSize/fontHeght无效
14    */
15    JCAPI.drawBarcode=function(content,x,y,width,height,rotation=0,fontSize,fontHeight,postion=2)

```

3.5.5 绘制二维码

代码块

```

1    /**

```

```

2  * 绘制二维码
3  * @param {string} content - 二维码内容
4  * @param {number} x - 起点x,单位mm
5  * @param {number} y - 起点y,单位mm (左上角坐标)
6  * @param {number} width - 宽,单位mm
7  * @param {number} height - 高,单位mm
8  * @param {number} rotation - 旋转角度,默认为0,支持0/90/180/270,旋转中心点左上角
9  * @param {number} ecc - 纠错等级 0-3
10 */
11 JCAPI.drawQRCode=function(content,x,y,width,height,rotation=0,ecc=2)

```

3.5.6 绘制线条

代码块

```

1  /**
2  * 绘制线条
3  * @param {number} x - 起点x, 单位mm
4  * @param {number} y - 起点y, 单位mm (左下角坐标)
5  * @param {number} width - 宽度, 单位mm
6  * @param {number} height - 高度, 单位mm
7  * @param {number} rotation - 旋转角度, 默认为0, 支持0/90/180/270, 旋转中心点左上角
8  *
9  * @description
10 * 计算线条的左上角为y坐标+线高
11 */
12 JCAPI.drawLine=function(x,y,width,height,rotation=0)

```

3.5.7 绘制矩形

代码块

```

1  /**
2  * 绘制矩形
3  *
4  * @param {*} x 起点x, 单位:毫米。
5  * @param {*} y 起点y, 单位:毫米。
6  * @param {*} width 对象宽, 单位:毫米。
7  * @param {*} height 对象高, 单位:毫米。
8  * @param {*} lineWidth 线条宽, 单位:毫米。
9  * @param {*} isFilled 是否填充, 缺省不填充
10 * @param {*} rotation 旋转角度, 缺省值为0, 支持0/90/180/270, 旋转中心点左上角。
11 */

```

```
12 JCAPI.drawRectangle=function(x,y,width,height,lineWidth,isFilled=false,rotation=0)
```

3.5.8 绘制图片

代码块

```
1  /**
2   * 绘制图片
3   * @param {string} path - 本地图片路径（暂不支持网络图片）
4   * @param {number} x 起点x,单位mm
5   * @param {number} y 起点y,单位mm
6   * @param {number} width 宽,单位mm
7   * @param {number} height 高,单位mm
8   * @param {number} rotation - 旋转角度，默认为0，支持0/90/180/270，旋转中心点左上角
9   *
10  * @description
11  * 暂不支持网络图片，需要将图片缓存到本地后，传入本地临时路径进行绘制
12  * 图像背景不可以设置为透明色，否则会打印出黑块
13  */
14 JCAPI.drawImage = function (path, x, y, width, height, rotation = 0)
```

3.5.9 取消正在打印的打印任务

代码块

```
1  /**
2   * 取消正在打印的打印任务
3   *
4   * @param callback 回调取消成功。
5   */
6  JCAPI.cancelPrint=function(callback)
```

3.5.10 开启打印任务

代码块

```
1  /**
2   * 开启打印任务
3   *
4   * @param gapType 设置打印的纸张类型，缺省1-间隙纸。2-黑标纸 3-连续纸 5-透明纸，
5   * @param darkness 设置该次打印浓度，B3S、B1、B203、B21、M2、K3、K3W浓度范围1-5 默认3，Z401、B32浓度范围1-15，默认8
```

```

6    * @param totalCount 设置总打印份数，表示所有页面的打印份数之和。例如，如果你有3页需要
    打印，
7    * 第一页打印3份，第二页打印2份，第三页打印5份，那么totalCount的值应为10（3+2+5）。
8    * @param callback 回调参数设置是否成功。
9    */
10   JCAPI.startJob=function(gapType,darkness,totalCount,callback)

```

3.5.11 提交打印任务准备打印

代码块

```

1   /**
2    * 打印
3    * @param onePageCount 设置当前页的打印份数，如果需要打印3页，第一页打印3份，第二页打印
    2份，
4    * 第三页打印5份，那么在3次提交数据时onePageCount的值分别为3，2，5
5    * @param callback 回调，表示可以发送下一页数据，不表示已打印完成，打印完成监听页码变化状
    态
6    * @param {JCSDKPrintOptionParas} optionPara 可选参数
7    */
8   JCAPI.print=function(onePageCount=1,callback,optionPara=null)

```

3.6 图片打印相关

3.6.1 图片打印

代码块

```

1   let JCSDKPrintImageParas = {
2     /**
3      * 打印份数
4      * @type {number}
5      * @default 1
6      */
7     count: 1,
8     /**
9      * 打印内容
10     * @type {string}
11     * @description base64数据(不包含data:image/png;base64,前缀)或者图片URL
12     * @required
13     */
14     data: "image/base64",
15     /**
16     * 画布组件ID

```

```
17     * @type {string}
18     * @required
19     */
20     canvasId: "test",
21     /**
22     * 画布组件实例
23     * @type {Object}
24     * @required
25     */
26     compent: Object,
27     /**
28     * 内容旋转角度
29     * @type {number}
30     * @default 0
31     * @enum {0|90|180|270}
32     */
33     orientation: 0,
34     /**
35     * 打印标签宽度(mm)
36     * @type {number}
37     * @default 50
38     */
39     width: 50,
40     /**
41     * 打印标签高度(mm)
42     * @type {number}
43     * @default 30
44     */
45     height: 30,
46     /**
47     * 图片左偏移量(像素)
48     * @type {number}
49     * @default 0
50     */
51     x: 0,
52     /**
53     * 图片上偏移量(像素)
54     * @type {number}
55     * @default 0
56     */
57     y: 0,
58     /**
59     * 打印区域宽度(mm)
60     * @type {number}
61     * @default 50
62     */
63     sWidth: 50,
```



```

64  /**
65   * 打印区域高度(mm)
66   * @type {number}
67   * @default 30
68   */
69  sHeight: 30,
70  /**
71   * 成功回调函数
72   * @type {function}
73   */
74  success: Object,
75  /**
76   * 失败回调函数
77   * @type {function}
78   */
79  fail: Object,
80  /**
81   * 完成回调函数(无论成功/失败)
82   * @type {function}
83   */
84  complete: Object,
85  /**
86   * EPC编码(B32R打印机支持)
87   * @type {string}
88   * @default ''
89   */
90  epc: '',
91  /**
92   * 是否使用虚线边框(H1S打印机支持)
93   * @type {boolean}
94   * @default false
95   */
96  hasDash: false,
97  /**
98   * 打印质量模式
99   * @type {number}
100  * @default 0
101  * @description 0-高质量打印, 1-高速度打印(部分机型支持)
102  */
103  speedOrQuality: 0
104 }
105 /**
106  * 提交图片URL或base64数据进行打印
107  * @function printImage
108  * @memberof JCAPI
109  * @description 为了更好的用户体验, 回调仅表示缓存成功/失败, 不影响已缓存的数据, 并不阻
断已有的打印任务

```

```

110  * @param {JCSDKPrintImageParas} para - 打印参数配置对象
111  * @property {number} para.count - 打印份数
112  * @property {string} para.data - 图片数据(base64或URL)
113  * @property {string} para.canvasId - 画布组件ID
114  * @property {Object} para.compent - 画布组件实例
115  * @property {number} [para.orientation=0] - 旋转角度(0/90/180/270)
116  * @property {number} [para.width=50] - 标签宽度(mm)
117  * @property {number} [para.height=30] - 标签高度(mm)
118  * @property {number} [para.x=0] - 图片左偏移(像素)
119  * @property {number} [para.y=0] - 图片上偏移(像素)
120  * @property {number} [para.sWidth=50] - 打印区域宽度(mm)
121  * @property {number} [para.sHeight=30] - 打印区域高度(mm)
122  * @property {function} [para.success] - 成功回调
123  * @property {function} [para.fail] - 失败回调
124  * @property {function} [para.complete] - 完成回调
125  * @property {string} [para.epc=''] - EPC编码(B32R支持)
126  * @property {boolean} [para.hasDash=false] - 是否使用虚线边框(H1S支持)
127  * @property {number} [para.speedOrQuality=0] - 打印质量模式(0-高质量,1-高速度)
128  */
129  printImage(para) {
130      manager.printImage(para);
131  },

```

图片打印流程如下：设置打印进度监听->设置打印异常监听->开启打印任务-提交图像数据->等待打印进度回调->打印完成

批量打印循环此流程，必须等打印进度回调完成后才可以执行下一次任务

3.7 回调相关

3.7.1 打印成功页码回调

代码块

```

1  /**
2   * 获取打印过程中，打印失败的错误上报
3   *
4   * @param callback 回调页码，count为页码，tid为B32R机型写入RFID标签是标签返回的tid数据
5   * {'count':int必需,'tid':string非必需}
6   */
7  JCAPI.didReadPrintCountInfo=function(callback)

```

3.7.2 获取打印过程中，打印失败的错误上报

代码块

```
1  /**
2   * 获取打印过程中，打印失败的错误上报
3   *
4   * @param callback 回调错误。
5   */
6  JCAPI.didReadPrintErrorInfo=function(callback)
7
8  let SDKErrorCode = {
9      "JCSDK_PRINT_ERR_TIMEOUT":{"errCode":0xff00,"msg":"打印超时"},
10     "JCSDK_PRINT_ERR_DISCONNECT":{"errCode":0xff01,"msg":"打印机断开"},
11     "JCSDK_PRINT_ERR_DATA":{"errCode":0xff02,"msg":"数据错误"},
12     "JCSDK_PRINT_ERR_BUSY":{"errCode":0xff03,"msg":"打印机忙碌"},
13     "JCSDK_PRINT_ERR_GetDATA":{"errCode":0xff04,"msg":"画布获取数据错误"},
14     "JCSDK_PRINT_ERR_SetPaperFailed":{"errCode":0xff05,"msg":"设置纸张失败错误"},
15     "JCSDK_PRINT_ERR_PrinterNotWork":{"errCode":0xff06,"msg":"查询打印纸张状态异常"},
16     "JCSDK_PRINT_ERR_0x01":{"errCode":0xff07,"msg":"盒盖打开"},
17     "JCSDK_PRINT_ERR_0x02":{"errCode":0xff08,"msg":"缺纸"},
18     "JCSDK_PRINT_ERR_0x03":{"errCode":0xff09,"msg":"电量不足"},
19     "JCSDK_PRINT_ERR_0x04":{"errCode":0xff0a,"msg":"电池异常"},
20     "JCSDK_PRINT_ERR_0x05":{"errCode":0xff0b,"msg":"手动停止"},
21     "JCSDK_PRINT_ERR_0x06":{"errCode":0xff0c,"msg":"数据错误"},
22     "JCSDK_PRINT_ERR_0x07":{"errCode":0xff0d,"msg":"温度过高"},
23     "JCSDK_PRINT_ERR_0x08":{"errCode":0xff0e,"msg":"出纸异常"},
24     "JCSDK_PRINT_ERR_0x09":{"errCode":0xff0f,"msg":"纸张错误"},
25     "JCSDK_PRINT_ERR_0x0a":{"errCode":0xff10,"msg":"rfid写入失败"},
26     "JCSDK_PRINT_ERR_0x0b":{"errCode":0xff11,"msg":"出纸异常"},
27     "JCSDK_PRINT_ERR_0x0c":{"errCode":0xff12,"msg":"发送epc指令但未检测到RFID标签"},
28     "JCSDK_PRINT_ERR_0x0d":{"errCode":0xff13,"msg":"检测到RFID标签但未发送EPC指令"},
29     "JCSDK_PRINT_ERR_0x0e":{"errCode":0xff14,"msg":"缺碳带"},
30     "JCSDK_PRINT_ERR_0x0f":{"errCode":0xff15,"msg":"打印头未锁紧"},
31     "JCSDK_PRINT_ERR_0x11":{"errCode":0xff16,"msg":"碳带已用完"},
32     "JCSDK_PRINT_ERR_0x12":{"errCode":0xff17,"msg":"碳带不匹配"},
33     "JCSDK_PRINT_ERR_epc":{"errCode":0xfe01,"msg":"epc格式不对"},
34     "JCSDK_PRINT_ERR_Drawing_Timeout":{"errCode":0xfe00,"msg":"绘制元素超时错误"}
35  };
```